YISHUN INNOVA JUNIOR COLLEGE
JC 1 PROMOTIONAL EXAMINATION
Higher 2

CANDIDATE NAME			
CG		INDEX NO	

COMPUTING **9569/02**

Paper 2 Practical **5 October 2026**

1 hour 50 minutes

Additional Materials: Removable storage device with the following files:

- `PROMO_template.ipynb`
- `SCORES.txt`
- `module.py`

READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

The number of marks is given in brackets [ ] at the end of each task.
The total number of marks for this paper is **60**.

Instruction to candidates:

Your program code and output for all the questions should be saved in a single file and downloaded as

PROMO_<your class>_<your name>.ipynb

For each of the sub-tasks, add a comment statement, at the beginning of the code using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:
```
#Task 1.1
Program Code
```

Output:

In [2]:
```
#Task 1.2
Program Code
```

Output:

In [3]:
```
#Task 1.3
Program Code
```

Output:

© YIJC

1 An organiser charges a registration fee for a community sports event based on the participant's age:

- the full registration fee of $18.00 for a participant aged from 16 to 65 years, inclusive
- a 25% discount for a participant aged from 5 to 15 years, inclusive, and
- free admission for a participant below 5 years or above 65 years.

Up to five participants can register as a group.

Task 1.1

Write program code that allows a participant to enter the following information:

- name of the contact person
- number of participants, and
- age of each participant.

Your program code should validate the user inputs for the contact person's name, the number of participants and each participant's age.

- The contact person's name must not be empty and must not contain digits. For example, `Devi d/o Rani` is a valid name, but `Devi123` is not.
- Each participant's age should be entered separately, one at a time, and stored in a list. Continue requesting the participants ages until the required number of valid ages has been entered.

Run the program code using the following information:

- name of the contact person: `Devi d/o Rani`
- number of participants: `4`
- age of each participant: `5, 16, 15, 66` [4]

Task 1.2

Write program code to calculate the total registration cost using the user inputs.

The website platform charges 3% of the total registration cost before adding the 10% Goods and Services Tax (GST). Only the final cost is rounded to two decimal places.

Display an output similar to the following for the user to check:

```
COMMUNITY EVENT REGISTRATION
Contact person: Devi d/o Rani
Number of participants: 4
Participant ages: [5, 16, 15, 66]
Total Cost: $50.99 (incl. 3% platform fee and 10% GST)
```

[2]

© YIJC **[Turn over**

2 The file `SCORES.txt` contains the results for the competitors in an inter-school diving competition. The headers on the first line are:

- `CompID`: the competitor's ID
- `Name`: the competitor's name
- `JC`: the school which the competitor is representing
- `S1`, `S2`, …, `S7`: scores awarded by the seven judges

The first three lines in the text file are shown below:

```
CompID,Name,JC,S1,S2,S3,S4,S5,S6,S7
D101,Alyssa Goh,ACJC,7.4,8.2,7.1,8.2,7.8,7.6,7.9
D102,Cheryl Lim,ACJC,6.3,7.1,7.6,5.9,6.8,6.5,7.3
```

Task 2.1

Write program code for the function `read_scores(filename)` to read the text file `filename` and return a list of tuples in the format `(CompID, S1, S2, S3, S4, S5, S6, S7)`, as shown below: [4]

```
[
    ("D101", 7.4, 8.2, 7.1, 8.2, 7.8, 7.6, 7.9),
    ("D102", 6.3, 7.1, 7.6, 5.9, 6.8, 6.5, 7.3),
    # further competitor records
]
```

The result for each competitor is calculated by removing the highest and lowest scores, then computing the average of the remaining five scores, rounded to two decimal places.

The functions `min()` and `max()` may be used to find the highest and lowest scores.

Task 2.2

Write program code for the function `calculate_results(competitors)` to accept the parameter `competitors`, which is the list of tuples obtained in **Task 2.1**, and compute and return the results as a list of tuples, as shown below: [3]

```
[
    ("D101", 7.78),
    ("D102", 6.8),
    # further competitor results
]
```


© YIJC

3 The Collatz conjecture states that a sequence starting with a positive integer `n` will eventually reach `1` when the following rules are repeatedly applied to `n` and each subsequent value:

- If `n` is even, the next value is `n` divided by `2`.
- If `n` is odd and greater than `1`, the next value is `3n + 1`.
- The sequence ends when `n` is `1`.

For example, starting with `6` produces the sequence:

`6 → 3 → 10 → 5 → 16 → 8 → 4 → 2 → 1`

The peak value is the largest value encountered in the sequence. Therefore, the peak value of the sequence starting with `6` is `16`.

Task 3

Write program code using recursion for the function `collatz(n)` to return the peak value in the sequence starting with `n`. [5]

4 A restaurant wants to use an app that allows customers to place their food orders. The orders will then be submitted to the kitchen for preparation.

The programmer uses a Queue Abstract Data Type (ADT) implemented using a Python list.

The file `module.py` contains the following provided methods for the `Queue` ADT:

- `make_empty_queue()` creates and returns an empty queue.
- `is_empty_queue(queue)` returns `True` if `queue` is empty and `False` otherwise.
- `queue_size(queue)` returns the number of items in `queue`.

The following tuple contains three food orders:

```
orders = ("F101", "F102", "F103")
```


© YIJC **[Turn over**

Task 4.1

Write program code for the function `order(queue, item)` to:

- add `item` to `queue`;
- display a message in the following format; and
- return `queue`.

```
Order F101 is added.
```

Write program code to create an empty queue named `food_queue` and use the function `order(queue, item)` to add each item in `orders` to `food_queue`. [3]

Task 4.2

Write program code for the function `complete(queue)` to allow the kitchen to indicate that an order has been prepared.

If `queue` is not empty, the function must:

- remove an item from `queue`;
- display a message in the following format; and
- return `queue`.

```
Order F101 is completed.
```

The function will return `None` if the queue is empty. [3]

Occasionally, an order may be accidentally marked as completed before the food has been prepared. Such an order must be returned to the front of the queue so that it is the next order to be prepared.

Task 4.3

Write program code for the function `undo(queue, item)` to:

- add `item` to `queue` and ensure that the order corresponding to `item` will be the next to be prepared;
- display a message in the following format; and
- return `queue`. [2]

```
Order F101 is returned to the queue.
```


© YIJC

5 An airline company has a fleet of aircraft, each with a unique name. The number of flights completed by each aircraft is recorded in the following list of tuples:

```
fleet = [
    ("Blue Horizon", 95),
    ("Cloud Nine", 156),
    ("Dream Chaser", 141),
    ("Good Vibes", 95),
    ("High Flyer", 72),
    ("Sky Dancer", 128),
    ("Sunny Side Up", 84),
    ("Wanderlust", 103)
]
```

Task 5

Write program code for the function `bubble_sort(lst)` to perform an **optimised**, **stable** and **in-place** bubble sort on `lst`, in ascending order of the number of flights completed.

Do not use Python's built-in functions `sort()` or `sorted()`. [4]

6 Riverdale Wildlife Park stores the names of eight animals in the list `animal_names`.

```
animal_names = [
    "Meerkat", "Zebra", "Otter", "Giraffe",
    "Panda", "Lion", "Flamingo", "Rhinoceros"
]
```

Task 6.1

Write program code for the helper function `merge(left, right)`. The parameters `left` and `right` are lists of strings that are already sorted in descending alphabetical order. The function returns a new list containing all strings from both lists, also in descending alphabetical order. Do not use `sort()`, `sorted()` or `reverse()` in this question. [2]

© YIJC **[Turn over**

The following partially completed pseudocode represents the recursive function `merge_sort(seq)`. It recursively splits a list `seq` into two sublists until each sublist contains at most one item.

Task 6.2

Complete the pseudocode by filling in blanks A, B and C. [3]

```
FUNCTION merge_sort(seq)
    IF ____________<A>_____________________ THEN
        RETURN seq
    ENDIF

    mid ← LENGTH(seq) DIV 2

    left ← _____________<B>_____________________
    right ← ____________<C>_____________________

    RETURN merge(left, right)
ENDFUNCTION
```

Task 6.3

Using your completed pseudocode from **Task 6.2** and the helper function `merge(left, right)` from **Task 6.1**, write program code for the function `merge_sort(seq)`.

Test your program code for `merge_sort(seq)` using the list `animal_names`. [2]

© YIJC

7 A football league stores each team's name and the number of goals it has scored as a tuple. The following list contains 20 such tuples, each with a unique name.

```
teams = [
    ("Woolwich FC", 100), ("Aston Village", 96),
    ("Bournemousse", 92), ("Brentwood Wanderers", 89),
    ("Brighttown & Hove", 85), ("Burntley", 81),
    ("ChelsTea", 77), ("Crystal Castle", 73),
    ("Evertonight", 69), ("Full Ham", 65),
    ("Leaders United", 61), ("Liver Pull FC", 57),
    ("Manchester Village", 53), ("Manchester Town", 49),
    ("Oldcastle United", 45), ("Nottingham Forrest", 41),
    ("Sunderlane", 36), ("London Hamsters", 31),
    ("West Hamster United", 25),
    ("Wolverhampton Wonderers", 20)
]
```

Each team's name is used to generate a hash index to insert the tuple into a hash table of size `25`.

Task 7.1

Write program code for the function `hash_name(item)`, which calculates and returns the hash index for the string `item`. The function must do the following:

- initialise `total` to `0`
- for each character in the string `item`, multiply its ASCII value by its position and add the result to `total`; the leftmost letter in the string, at index `0`, has position `1`, and
- compute and return the hash index for a table of size `25`. [3]

© YIJC **[Turn over**

The team tuples are stored in a hash table implemented using a Python list. A collision occurs when different team names produce the same hash index.

Task 7.2

Write program code to create a hash table named `team_table` of size 25 and insert every tuple in `teams`, in the given order.

Each cell must contain:

- an empty string when unused;
- a single team tuple when there is only one team; or
- a list of team tuples when there is more than one team.

Use the team name from each tuple as the argument to `hash_name()` to obtain its hash index. [3]

The filled hash table can be searched using a team's name to retrieve its number of goals scored.

Task 7.3

Write program code for the function `find_goals(table, team_name)` that searches the hash table for a team and returns the number of goals scored. Return `None` if the team is not found. [2]

© YIJC

8 A football league identifies each team using a unique three-letter team code. The team codes are stored in a Binary Search Tree (BST) Abstract Data Type (ADT).

The file `module.py` contains the following methods for the BST ADT:

- `make_empty_tree()` returns an empty tree
- `make_tree(entry, left, right)` returns a tree containing `entry` element, the left sub-tree and the right sub-tree
- `entry(tree)` returns the element at the root node of the tree `tree`
- `left_branch(tree)` returns the left subtree of `tree`
- `right_branch(tree)` returns the right subtree of `tree`
- `is_empty(tree)` returns `True` if `tree` is empty; otherwise, it returns `False`
- `print_tree(tree)` displays the entries in `tree` one level at a time.

The following list contains the unique team codes:

```
team_codes = [
    "LIV", "OLD", "CHE", "FUL", "MNV",
    "BRE", "WOL", "EVE", "MNT", "BOU",
    "WHU", "LEE", "NOT", "BUR", "LON",
    "CRY", "SUN", "BRI", "WOO", "AST"
]
```

Task 8.1

Write program code for the recursive function `insert_item(tree, item)` to insert `item` into its correct position in the BST `tree`. The function returns the resulting BST. [3]

Task 8.2

Write program code for the recursive function `search_tree(tree, target)` to search for the element `target` in the BST `tree`. It returns `True` if `target` is found, and `False` otherwise. [3]

Task 8.3

Write program code for the recursive function `traversal(tree)` to perform an in-order traversal on the BST `tree`, and return the entries in a list. The function returns an empty list if `tree` is empty. [2]

© YIJC **[Turn over**

9 A social media agency analyses the hashtags used on various social media platforms. Some of the hashtags collected are stored in the following list:

```
hashtags = [
    "#Travel", "#AI2026", "#Food", "#travel", "#Viral",
    "#ai2026", "#Tech", "#FOOD", "#SocialMedia", "#viral",
    "#Travel", "#TECH", "Lifestyle", "##Trending", "#Sale!",
    "#", "#Social Media", "#AI2026", "#socialmedia", "#Food"
]
```

A valid hashtag, denoted with a single `#` symbol character:

- contains only letters and digits after `#`, in any combination, and
- has 2 to 14 characters, excluding the `#` symbol character.

Task 9.1

Write program code for the function `is_valid(tag)`, which returns `True` if `tag` is a valid hashtag and returns `False` otherwise. [2]

Test your program code with two test cases. [1]

The agency uses a dictionary to track the number of occurrences of each valid hashtag. Hashtags are case-insensitive; for example, `#Travel` and `#travel` are accepted as the same hashtag.

Task 9.2

Write program code for the function `compile_hashtags(lst)` to process a list of hashtags `lst`. Your function must:

- ignore all invalid hashtags
- use each valid lowercase hashtag as a dictionary key and store its number of occurrences as the corresponding value, and
- return the dictionary.

Test your program using the list `hashtags`. [4]

END OF PAPER

© YIJC